

CSE235 Week 3: If Statements and Boolean Logic

This week we will discuss if statements, which are conditional statements that execute a code block only if the provided condition is true. We will also discuss Boolean logic, which is used to construct the conditions for conditional statements.

If Statements

Below is the basic syntax for an if statement in Python:

```
if condition:  
    statement
```

The if statement starts with the keyword `if`, followed by a condition that determines if statements in the block should be run. In Python, the condition for an if statement does not need to be in parentheses. The condition is ended by a colon. The if statement must contain at least one statement. Remember that in Python, blocks are denoted by indentation, so the statement(s) in the body of the if are indented. If the body of the if only has one statement, there is a shorthand syntax where the statement is placed on the same line as the if:

```
if condition: statement
```

If statements have an optional else clause, which only executes when the provided condition is false:

```
if condition:
    statement
else:
    statement
```

Note that the `else` keyword is at the same level of indentation as its corresponding if statement. The `else` is followed by a colon. Just like the if statement, the body of the else clause must be indented. Below is a basic example of an if-else statement in use:

```
# read an integer from the user
x = int(input("Enter an integer: "))

# remember that % returns the remainder
# therefore, if the remainder of dividing
# a number by 2 is 0, the number is even
if x % 2 == 0:
    print("The number you entered is even")
else:
    print("The number you entered is false")
```

In the case that multiple conditions need to be checked, there is also the `elif` keyword. `elif` stands for "else if". If the condition for the first if statement is false, the condition for the elif is checked next. There can only be one `if` and one `else` in a single if statement, but there can be arbitrarily many `elif`s.

```
x = int(input("Enter an integer: "))

# 2 * 3 is 6, so if the remainder of dividing by 6 is 0
# the number is divisible by both 2 and 3
if x % 6 == 0:
    print("The number you entered is divisible by 2 and 3")
# it's not divisible by both 2 and 3, but may be divisible by
# one or the other
elif x % 2 == 0:
    print("The number you entered is divisible by 2, but not 3")
elif x % 3 == 0:
    print("The number you entered is divisible by 3, but not 2")
# if we reach this point, the number was not divisible by either
else:
    print("The number you entered is not divisible by 2 nor 3")
```

Boolean Logic

Boolean logic, also called Boolean algebra, are expressions that evaluate to either true or false. It is named for the mathematician George Boole. There are three main operations in Boolean logic: AND, OR, and NOT. The AND operation is a binary operation and is true iff (if and only if) both operands are true. The OR operation is a binary operation and is false iff both operands are false. The NOT operation is a unary operation, and simply negates its operand. I.E., NOT true is false and NOT false is true.

Truth Tables

Truth tables are used to visualize the possible outcomes of Boolean operations. Below is a truth table for two values, a and b, and the possible combinations of them:

								NOT	
				a	a	NOT	NOT	a	NOT
				AND	OR	(a	(a	AND	a OR
a	b	NOT	NOT	b	b	AND	OR	NOT	NOT
true	true	false	false	true	true	false	false	true	true
true	false	false	true	false	true	true	false	false	true
false	true	true	false	false	true	true	false	false	true
false	false	true	true	false	false	true	true	true	true

There should be a column for each input variable and operation between the input variables. The number of rows scales exponentially with the number of input variables. In this truth table, there are two input variables, a and b. a and b can have two possible values, true or false, and there are two input variables, so the number of rows is $2^2 = 4$. If we added a third input variable, we'd have $2^3 = 8$ rows. For four input variables, we'd have $2^4 = 16$ rows, and so on. Therefore, truth tables scale poorly when there are many input variables, but if you can minimize the number of input variables truth tables can be useful for determining the results of various Boolean expressions.

Precedence

Comparison operators have lower precedence than arithmetic operators. Arithmetic operators (+, -, *, /, etc.) follow the PEMDAS you should have learned in grade school. Next, the comparison operators are applied. The <, >, <=, >= are applied before == and !=. Next, AND is applied. Finally, OR is applied. This is true in most programming languages.

De Morgan's Laws

De Morgan's laws are a pair of equivalences used for simplifying boolean expressions. We'll skip the formal proof, and instead look to the truth table to demonstrate the validity of the laws. 1. NOT (a AND b) = (NOT a) OR (NOT b) 2. NOT (a OR b) = (NOT a) AND (NOT b) If you look at the table, you'll see that for each row, the column for "NOT (a AND b)" has the same value as "NOT a OR NOT b". The same applies for "NOT (A OR B)" and "NOT a AND NOT b".

In modern programming, these are less relevant, as the compiler/interpreter will apply De Morgan's laws to make Boolean logic more efficient on its own. Nonetheless, remembering the laws can be useful when trying to think through what a Boolean expression is doing.

Numbers as Boolean Values

Many modern programming languages are based on C. C did not have a Boolean type originally. Instead, it used 0 for false and 1 for true. In reality, any numeric value that is not 0 is interpreted as true. The same is true in Python. For example:

```
if 5:
    print("5 is true!")
```

The same is true for floating point numbers:

```
if -4.2:
    print("-4.2 is true!")
```

If Statement Examples

Below are example programs using if statements.

Login

```
# obviously, this isn't accurate to a real login system
# it's just to demonstrate if statements

# read username and password
username = input("Enter username: ")
password = input("Enter password: ")

# if the username is admin and password is correct
if username == "admin" and password == "password":
    print("Admin login successful")
# if username is admin but password is wrong
elif username == "admin":
    print("Admin login unsuccessful. Incorrect password.")
# if username is user and password is correct
elif username == "user" and password == "password":
    print("User login successful")
# if username is user but password is wrong
elif username == "user":
    print("Admin login unsuccessful. Incorrect password.")
# any other username is invalid
else:
    print("Unknown user:", username)
```

Vending Machine

```
# print out the options and prices
print("A1: Plain Chips $1.50 -----",
      "A2: BBQ Chips $1.50 -----" ,
      "A3: Jalepeno Chips $1.50")
print("B1: PB Cups $1.75 -----",
      "B2: Chocolate Bar $2.50 ---",
      "B3: Cookie $2.00")
print("C1: Trail Mix $2.25 -----",
      "C2: Protein Bar $3.50 -----",
      "C3: Granola Bar $2.50")

# get money from user
money = float(input("Enter money: $"))

# read the row and column of the selection
row = input("Enter row: ")
column = int(input("Enter column: "))

# used to store the item price to compare later
item_price = 0

# check the rows first
```

```

# allow uppercase or lowercase letters
if row == "A" or row == "a":
    # then check the column values
    if column == 1:
        # set the price
        item_price = 1.50
    elif column == 2:
        item_price = 1.50
    elif column == 3:
        item_price = 1.50
    # otherwise, print that the column is invalid
    else:
        print("Invalid column")

elif row == "B" or row == "b":
    if column == 1:
        item_price = 1.75
    elif column == 2:
        item_price = 2.50
    elif column == 3:
        item_price = 2.00
    else:
        print("Invalid column")

elif row == "C" or row == "c":
    if column == 1:
        item_price = 2.25
    elif column == 2:
        item_price = 3.50
    elif column == 3:
        item_price = 2.50
    else:
        print("Invalid column")

else:
    print("Invalid row")

# if the item_price is still 0, the selection was invalid
if item_price == 0:
    print("Invalid selection, try again later")
# check if the amount of money entered is more than the price
elif item_price < money:
    change = money - item_price
    print("Your change is $", change)
# otherwise, they can't afford it
else:
    shortage = item_price - money
    print("You are $", shortage, "short")

```

Conclusion

This week we discussed if statements. If statements are conditional statements that execute a block of code only if their condition is true. If statements can have an optional else clause, which executes a block of code if the condition is false. There are also "else if" statements, denoted by the `elif` keyword, that allow multiple conditions to be used within the same if statement. We also discussed Boolean logic for constructing complex conditions.